

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA0609	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	10% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	JEEVA	Reg. No.:	192421334
Section / Batch:	Information Technology	Date:	25/08/2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Analyze the given questions thoroughly before answering.
- Compare multiple aspects such as methods, results, advantages, and limitations clearly.
- Critically evaluate the strengths, weaknesses, and implications with proper justification.
- Present meaningful insights, interpretations, and well-supported conclusions from the comparisons.

Question Type	Focus Area	Questions
Comparative Analysis	Comparative analysis of Dynamic Programming algorithms (Dice Throw Problem vs Coin Change) for optimization problems.	Q1

SECTION A – Comparative Analysis

Q13. Dice Throw Problem vs Coin Change (DP Comparison)

Given number of dice and target sum, solve the Dice Throw Problem and compare it with Coin Change DP formulation. Analyze similarities in state transitions and subproblem overlap. Identify differences in constraints and recurrence relations. Evaluate computational complexity of both problems. Discuss scalability for larger inputs. Generate insights on DP problem modeling techniques. Instructions: Solve the assigned problem, show the algorithm/steps clearly, analyze correctness, time and space complexity, and provide the requested comparison or insights.

Code of Conduct

I Jeeva (192421334) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

RUBRICS FOR CALCULATION

Comparative Analysis					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Selection of Studies/Parameters	20%	Selects highly relevant and diverse studies with well-defined comparison parameters	Selects relevant studies with minor gaps in parameter definition	Limited or partially relevant selection	Poor or inappropriate selection of studies
Comparison & Differentiation	25%	Clearly compares multiple aspects (methods, results, limitations) with strong differentiation	Good comparison with minor gaps	Basic comparison with limited depth	No clear comparison; mostly descriptive
Critical Evaluation	25%	Critically evaluates strengths, weaknesses, and implications with strong justification	Provides evaluation with some justification	Limited evaluation; lacks depth	No critical evaluation provided
Synthesis & Insight Generation	20%	Generates meaningful insights and conclusions from comparisons	Some insights derived from comparison	Limited or unclear insights	No insights generated
Clarity	10%	Information is clearly presented (tables/figures) with logical structure	Generally clear with minor issues	Presentation lacks clarity or structure	Poor presentation and difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Selection of Studies/Parameters	20%				
Comparison & Differentiation	25%				
Critical Evaluation	25%				
Synthesis & Insight Generation	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

DICE THROW PROBLEM VS COIN CHANGE — DP COMPARISON

Both the Dice Throw Problem and the Coin Change Problem are classic Dynamic Programming problems built around composing a target sum from smaller unit values. They share the same underlying idea of "building up a sum step by step" but differ in how the steps are constrained.

1. Problem Overview

Feature	Dice Throw Problem	Coin Change Problem
Problem type	Counting / combinatorial optimization	Counting / optimization (min coins)
Input	Number of dice (n), faces per die (m), target sum (X)	Set of coin denominations, target sum (X)
Goal	Count number of ways to obtain sum X using exactly n dice	Count ways to form sum X, or find minimum coins needed
Resource usage	Exactly n dice must be used (fixed count)	Unlimited supply of each coin (unbounded count)
Per-step choice	Any value from 1 to m	Any coin from the given denomination set
Main data structure	2D DP table (dice used × sum)	1D DP array (sum), indexed per coin
Output	Number of ways to reach sum X with n dice	Number of combinations, or minimum coin count

2. Dice Throw Problem

DP Formulation

- State: $dp[i][j]$ = number of ways to obtain sum j using exactly i dice.
- Base case: $dp[0][0] = 1$ (zero dice give sum zero in exactly one way); $dp[0][j] = 0$ for $j > 0$.
- Recurrence: $dp[i][j] = \sum dp[i-1][j-f]$ for $f = 1$ to m , whenever $(j-f) \geq 0$.
- Answer: $dp[n][X]$.

Steps (Example: n = 2 dice, m = 6 faces, target X = 7)

$dp[1][k] = 1$ for $k = 1..6$ (a single die shows each face exactly once)
 $dp[2][7] = dp[1][6] + dp[1][5] + dp[1][4] + dp[1][3] + dp[1][2] + dp[1][1]$
 $= 1 + 1 + 1 + 1 + 1 + 1 = 6$

So there are 6 ways to roll a sum of 7 with two standard dice — e.g. (1,6), (2,5), (3,4), (4,3), (5,2), (6,1).

Correctness

Every way of reaching sum j with i dice must end with the i -th die showing some face f ($1 \leq f \leq m$), leaving the remaining $(i-1)$ dice to form the sum $(j-f)$. Summing $dp[i-1][j-f]$ over all valid f counts every such way exactly once, with no overlap and no omission, so the recurrence is both complete and non-redundant. Because $dp[i-1][*]$ is computed before $dp[i][*]$ and reused across many sums, the problem exhibits overlapping subproblems and optimal substructure, which is what makes the DP formulation both correct and efficient.

3. Coin Change Problem (Counting Combinations)

DP Formulation

- State: $dp[j]$ = number of ways to form sum j using the available coin denominations (each coin usable any number of times).
- Base case: $dp[0] = 1$ (one way to make sum zero — choose no coins).
- Recurrence: for each coin c in coins, for $j = c$ to X : $dp[j] += dp[j-c]$.
- Processing coins in the outer loop (rather than the sum) ensures combinations are counted, not permutations.

Steps (Example: coins = {1, 2, 3}, target X = 4)

$dp[0]=1, dp[1]=0, dp[2]=0, dp[3]=0, dp[4]=0$ (initial)

After coin 1: $dp = [1, 1, 1, 1, 1]$

After coin 2: $dp[2] += dp[0] \rightarrow 2, dp[3] += dp[1] \rightarrow 2, dp[4] += dp[2] \rightarrow 3$

After coin 3: $dp[3] += dp[0] \rightarrow 3$, $dp[4] += dp[1] \rightarrow 4$

Final $dp[4] = 4$, corresponding to the combinations $\{1,1,1,1\}$, $\{1,1,2\}$, $\{2,2\}$, $\{1,3\}$.

Correctness

Fixing the outer loop to iterate over coin denominations means that, for any coin c , the update $dp[j] += dp[j-c]$ only counts combinations in which coins are considered in a fixed, non-decreasing "order of introduction" — this prevents the same multiset of coins from being counted multiple times under different orderings. Each $dp[j]$ therefore represents a complete and non-redundant count of combinations, again relying on overlapping subproblems ($dp[j-c]$ is reused for many j) and optimal substructure.

4. Similarities in State Transitions and Subproblem Overlap

- Both problems build a solution for a target sum by combining the solution for a smaller sum with one additional "unit" (a die face or a coin).
- Both recurrences are additive: the count/optimum for state j is obtained by summing or minimizing over the counts/optima of several smaller states.
- Both exhibit heavy overlapping subproblems — the same smaller-sum values ($dp[j-f]$ or $dp[j-c]$) are reused repeatedly across different larger sums, which is exactly what tabulation exploits.
- Both have optimal/countable substructure: the way to reach sum j is fully determined by ways to reach smaller sums plus one local choice.
- Both can be implemented with straightforward bottom-up tabulation, filling a table of size proportional to (resource used $\times$ target sum).

5. Differences in Constraints and Recurrence Relations

Aspect	Dice Throw	Coin Change
Resource bound	Bounded and fixed — exactly n dice must be used	Unbounded — any number of coins may be used
State dimension	Two dimensions: dice used (i) and sum (j)	Usually one dimension: sum (j); coin denomination handled via loop order, not table dimension
Range of per-step choice	Choices form a contiguous range $1..m$	Choices are a fixed, often sparse, discrete set of denominations
Recurrence width	Sums over up to m terms per state (face values)	Sums/derives over the number of coin denominations (often fewer than m)
Ordering concern	Not applicable — each die corresponds to a distinct table row, so no double-counting risk	Loop order matters: coin-outer counts combinations; sum-outer would count ordered permutations
Optimization goal	Counting (number of ways)	Counting (ways) or minimization (fewest coins) — two different DP variants

A useful conceptual bridge: if the Coin Change problem were restricted to using exactly k coins (a bounded-count variant), its DP state would need an extra "coins used" dimension — becoming structurally identical to the Dice Throw table (uses-used $\times$ sum). The core difference, therefore, is really about bounded vs. unbounded resource use, not about the two problems being fundamentally unrelated.

6. Time and Space Complexity

Problem	Time Complexity	Space Complexity	Optimized Space
Dice Throw (n dice, m faces, target X)	$O(n \cdot X \cdot m)$	$O(n \cdot X)$	$O(X)$ with a rolling 1D array
Coin Change – counting ways (k coins, target X)	$O(k \cdot X)$	$O(X)$	$O(X)$ (already optimal, 1D array)
Coin Change – minimum coins (k coins, target X)	$O(k \cdot X)$	$O(X)$	$O(X)$

Dice Throw's inner sum over m face values can be reduced from $O(m)$ to $O(1)$ per state using a sliding-window / prefix-sum technique (maintaining a running window sum as j increases), bringing its overall time down to $O(n \cdot X)$ — matching the structural efficiency of Coin Change's single-loop-per-coin approach.

7. Scalability for Larger Inputs

- Dice Throw: as m (faces) grows, the naive recurrence's inner loop grows proportionally; the sliding-window optimization removes this dependency, so the dominant cost becomes $n \times X$ — scalable for large targets provided memory for the table (or a rolling array) is available.
- Dice Throw: as n (number of dice) grows, both time and the natural 2D table grow linearly in n ; a rolling 1D array keeps memory bounded to $O(X)$ regardless of n .
- Coin Change: scalability is governed mainly by the target sum X and the number of denominations k ; since k is typically small and fixed (e.g., currency systems), Coin Change tends to scale very well even for large X .
- Both problems become memory-sensitive only when X is very large; sparse/segmented tabulation or meet-in-the-middle style tricks can help in extreme cases for either problem.

8. Real-World Applications

Dice Throw / Combinatorial Sum-Counting DP

- Probability computation in board games and gambling (e.g., odds of rolling a given total).
- Risk and simulation modeling where outcomes are sums of independent bounded trials.
- Load/resource distribution counting problems in operations research.
- Cryptographic and combinatorial enumeration tasks involving bounded-value sequences.

Coin Change

- Currency exchange and cash-register "making change" systems.
- Vending machines and point-of-sale denomination optimization.
- Resource allocation and budgeting where a target must be composed from fixed-size units.
- Inventory decomposition and packaging problems (e.g., composing a shipment weight from fixed box sizes).

9. Key Insights on DP Problem Modeling

- Identify whether the resource used is bounded (fixed count, as in Dice Throw) or unbounded (unlimited supply, as in Coin Change) — this decides whether the DP state needs an explicit "count used" dimension.
- When per-step choices form a contiguous numeric range, recurrences that sum over that range are good candidates for sliding-window/prefix-sum optimization.
- When counting combinations rather than permutations, fix the iteration order of the "choice" dimension (e.g., coins) in the outer loop to avoid overcounting.
- Both problems illustrate that DP state design is really a translation of the problem's constraints (fixed vs. unlimited resource, contiguous vs. discrete choice set) into table dimensions and loop order.
- Overlapping subproblems and optimal/countable substructure are what justify tabulation over brute-force enumeration in both cases, turning exponential search spaces into low-order polynomial-time solutions.

10. Final Comparison Table

Parameter	Dice Throw Problem	Coin Change Problem
Problem Type	Counting (combinatorial DP)	Counting or minimization (DP)
Main Objective	Count ways to reach sum X with exactly n dice	Count combinations for sum X , or minimize coins used
Input	Number of dice, faces per die, target sum	Coin denominations, target sum
Resource Constraint	Fixed / bounded (exactly n picks)	Unbounded (unlimited coin supply)
DP State	$dp[i][j]$ — dice used $\times$ sum	$dp[j]$ — sum (coin handled via loop order)
Recurrence Basis	Sum over contiguous face range (1.. m)	Sum/min over discrete coin denominations
Data Structure	2D table (optimizable to rolling 1D array)	1D array
Time Complexity	$O(n \cdot X \cdot m)$, optimizable to $O(n \cdot X)$	$O(k \cdot X)$
Space Complexity	$O(n \cdot X)$, optimizable to $O(X)$	$O(X)$
Overcounting Risk	None — dice are distinct table rows	Yes if loop order is wrong (combinations vs. permutations)
Real-World Applications	Probability/odds computation, simulations, risk	Currency change-making, vending systems,

Parameter	Dice Throw Problem	Coin Change Problem
	modeling	budgeting
Core DP Property	Overlapping subproblems + optimal substructure	Overlapping subproblems + optimal substructure